

ÉCOLE NATIONALE SUPÉRIEURE POLYTECHNIQUE DE YAOUNDÉ

Département Génie Informatique

RAPPORT DE TRAVAUX PRATIQUES

ANI-IAN 4068 : Coder pour la VR, l'XR et l'AR II

De zéro à 11/11 :

**Le vrai parcours pour installer Jenga,
compiler Nkentseu et implémenter
11 TPs de maths 3D**

Auteur : TEZEMBON DJOUFFO Loïc

Matricule : 22P076

Encadreur : Mr. Teuguia

Date : 5 avril 2026

"Le meilleur moyen d'apprendre, c'est de se battre contre les erreurs."

Avant-propos

Ce rapport n'est pas un rapport classique.

Il ne va pas vous énumérer des définitions que vous trouverez dans n'importe quel manuel. Il va vous raconter ce qui s'est *réellement* passé — les commandes qui ont planté, les messages d'erreur incompréhensibles à première vue, les moments de doute, et la satisfaction quand enfin tout compile.

L'objectif de ces TPs était d'apprendre à utiliser **Jenga Build System** et le framework C++ **Nkentseu**, puis d'implémenter des briques mathématiques fondamentales pour le rendu 3D : arithmétique flottante, vecteurs, matrices, quaternions et animation.

Ce qui suit est le récit honnête de cette aventure.

Table des matières

Avant-propos	1
1 L’installation : plus compliquée que prévu	4
1.1 Première tentative : la méthode officielle	4
1.2 Clone depuis GitHub : les caprices du réseau	4
1.3 Le labyrinthe des dossiers	5
1.4 Les noms de fichiers : Linux est sensible à la casse	5
1.5 Le script de lancement cassé	6
2 Compiler Nkentseu : les dépendances graphiques	8
2.1 Fork et clone du projet	8
2.2 Bibliothèques X11 manquantes	8
3 Le Sandbox : le terrain de jeu des TPs	10
3.1 Structure du projet	10
3.2 Le fichier de tests : ALL_TP.cpp	10
4 TPs 1 à 3 : Plonger dans les flottants IEEE 754	12
4.1 TP1 : Regarder un flottant en face	12
4.1.1 Ce qu’on a appris	12
4.1.2 L’implémentation	12
4.1.3 Ce que les tests ont révélé	13
4.2 TP2 : Quand les additions deviennent mensongères	13
4.2.1 Le problème de la sommation naïve	13
4.2.2 L’algorithme de Kahan : compenser les erreurs	13
4.3 TP3 : 33 tests pour valider tout ça	14
5 TPs 4 à 6 : Des vecteurs en 2D, 3D et 4D	15
5.1 TP4 : Vec2d — les fondations	15
5.2 TP5 : Vec3d et Gram-Schmidt	15
5.2.1 Le produit vectoriel	15
5.2.2 Gram-Schmidt : construire une base orthonormée	16

5.3	TP6 : Vec4d et la première image 3D	16
6	TPs 7 à 9 : Matrices, Rasteriseur et TRS	17
6.1	TP7 : La matrice 4×4 et son inverse	17
6.2	TP8 : Le cube tourne!	17
6.3	TP9 : TRS et décomposition	18
7	TPs 10 et 11 : Quaternions et Animation	19
7.1	TP10 : Les quaternions	19
7.1.1	Pourquoi les quaternions?	19
7.1.2	Ce qui est testé	19
7.2	TP11 : SLERP vs LERP — animer proprement	20
7.2.1	Le problème de l'interpolation de rotations	20
7.2.2	SLERP : l'interpolation sphérique	20
7.2.3	Ce que le code produit	20
8	Bilan final : ce que j'en retiens	21
8.1	Résultats des tests	21
8.2	Les vraies leçons de ce projet	22

Chapitre 1

L'installation : plus compliquée que prévu

"Ça devrait prendre 5 minutes..."

— Moi, naïvement

1.1 Première tentative : la méthode officielle

Le README de Jenga dit clairement : *"pip install jenga-build-system"*. Simple, non ? Sauf que :

```
pip install jenga-build-system
```

```
ERROR: Could not find a version that satisfies the requirement  
jenga-build-system (from versions: none)  
ERROR: No matching distribution found for jenga-build-system
```

Le paquet n'existe tout simplement pas sur PyPI.

Pas de panique. Le README mentionne une méthode 2 : installer depuis GitHub. Direction le dépôt officiel.

1.2 Clone depuis GitHub : les caprices du réseau

```
git clone https://github.com/RihenUniverse/Jenga
```

Clone interrompu

```
error: RPC failed; curl 18 transfer closed with outstanding
read data remaining
fatal: early EOF
```

Connexion instable. Le téléchargement se coupe à mi-chemin.

Clone léger avec `--no-recurse-submodules`

```
git clone --no-recurse-submodules https://github.com/
RihenUniverse/Jenga
```

L'option `--no-recurse-submodules` évite de télécharger les sous-projets liés, ce qui allège considérablement le transfert.

1.3 Le labyrinthe des dossiers

Une fois Jenga cloné, il faut trouver le bon dossier pour l'installation. La structure est trompeuse : il y a un dossier `Jenga/` qui contient *encore* un dossier `Jenga/` à l'intérieur.

pip install depuis le mauvais dossier

```
ERROR: file:///home/loic/Jenga/Jenga does not appear to be
a Python project: neither 'setup.py' nor 'pyproject.toml' found.
```

Trop profond dans l'arborescence.

Remonter d'un niveau

```
cd ~/Jenga # le dossier externe, l o se trouve pyproject.
toml
pip install .
```

1.4 Les noms de fichiers : Linux est sensible à la casse

Après installation, jenga répond mais lève une erreur :

ModuleNotFoundError

```
ModuleNotFoundError: No module named 'Jenga.Core'
```

Sur Linux, `core` $\neq$ `Core`. Les dossiers et fichiers étaient en minuscules alors que Python les cherchait avec une majuscule.

Renommer tous les fichiers concernés

```
cd ~/Jenga/Jenga
mv core Core && mv utils Utils
cd Core && mv api.py Api.py && mv loader.py Loader.py
mv variables.py Variables.py && mv commands.py Commands.py
cd ../Utils && mv display.py Display.py
cd ../Commands
mv build.py Build.py && mv run.py Run.py && mv create.py Create.
py
# ... et tous les autres fichiers du dossier Commands
```

1.5 Le script de lancement cassé

Même après correction des noms, `jenga` ne se lance pas :

Mauvais nom dans jenga.sh

```
python3: can't open file '/home/loic/Jenga/Jenga/jenga.py':
[Errno 2] No such file or directory
```

Le fichier s'appelle `Jenga.py` (avec majuscule) mais le script `jenga.sh` cherchait `jenga.py`.

Corriger le script + PATH + alias

```
# Corriger le nom du fichier dans le script
sed -i 's/jenga.py/Jenga.py/g' ~/Jenga/Jenga/jenga.sh

# Rendre le script executable
chmod +x ~/Jenga/Jenga/jenga.sh

# Créer un alias permanent
echo 'alias jenga="$HOME/Jenga/Jenga/jenga.sh"' >> ~/.bashrc
source ~/.bashrc
```

Jenga enfin fonctionnel !

```
$ jenga
Jenga Build System v2.0.1
Usage: Jenga <command> [options]
  build, b      Compile le workspace
  run, r        Execute un projet
  test, t       Lance les tests unitaires
  ...
```

Le truc à retenir

Sur Linux, la casse des noms de fichiers est critique. Un fichier `Api.py` et `api.py` sont deux fichiers *différents*. Toujours vérifier avec `ls` après une opération de renommage.

Chapitre 2

Compiler Nkentseu : les dépendances graphiques

2.1 Fork et clone du projet

Nkentseu est le framework C++ dans lequel on va travailler. La bonne pratique est de *forker* le dépôt sur GitHub (créer sa propre copie) avant de cloner :

```
# Après avoir fork sur github.com/RihenUniverse/nkentseu
git clone https://github.com/Tezz937/nkentseu
cd nkentseu
jenga b --platform linux
```

2.2 Bibliothèques X11 manquantes

Erreur de compilation : X11/X.h introuvable

```
fatal error: X11/X.h: Aucun fichier ou dossier de ce nom
38 | #include <X11/X.h>
```

Le module Glad (bibliothèque OpenGL) a besoin des en-têtes X11 pour gérer les fenêtres graphiques sur Linux. Ils n'étaient pas installés.

Installer les dépendances graphiques

```
sudo apt-get install libx11-dev libxrandr-dev libxinerama-dev \
libxcursor-dev libxi-dev libgl1-mesa-dev
```

Compilation complète réussie

```
Projects Built: 33/33  
Time:          1m55.6s  
Status:        SUCCESS
```

Les 33 projets du workspace compilent sans erreur.

Le truc à retenir

Sur Ubuntu, quand une compilation échoue avec "fichier .h introuvable", c'est presque toujours une bibliothèque système manquante. La commande `apt-get install libXXX-dev` est ton amie.

Chapitre 3

Le Sandbox : le terrain de jeu des TPs

3.1 Structure du projet

Le dossier `Applications/Sandbox` est l'endroit où tous les TPs sont implémentés. Après remplacement par la version fournie :

```
Sandbox/  
  include/          ← fichiers d'en-tête communs  
  Sandbox.jenga     ← configuration de build  
  src/  
    main.cpp        ← point d'entrée  
    Float.cpp/.h    ← TP1, TP2, TP3  
    Vec2d.h         ← TP4  
    Vec3d.h         ← TP5  
    Vec4d.h         ← TP6  
    Mat4d.h         ← TP7, TP8, TP9  
    Quat.h          ← TP10, TP11  
    NKImage.h       ← utilitaire image  
  tests/  
    ALL_TP.cpp      ← tous les tests unitaires
```

3.2 Le fichier de tests : `ALL_TP.cpp`

La particularité de ce projet est que tous les tests sont regroupés dans un seul fichier. La syntaxe utilise le framework **Unitest** intégré à Nkentseu :

Listing 3.1 – Exemple de cas de test avec Unitest

```
1 TEST_CASE(Loic_TP3, ValidationFonctionsFloat) {  
2     // Test que NaN n'est pas une valeur finie valide  
3     ASSERT_TRUE(!isFiniteValid(std::numeric_limits<float>::quiet_NaN()  
4         ));  
5  
6     // Test qu'une valeur normale est bien valide  
7     ASSERT_TRUE(isFiniteValid(42.0f));  
8  
9     // ... 31 autres assertions  
10 }
```

Pour lancer uniquement les tests du Sandbox sans recompiler :

```
jenga test --no-build --project Sandbox_Tests
```

Chapitre 4

TPs 1 à 3 : Plonger dans les flottants IEEE 754

4.1 TP1 : Regarder un flottant en face

4.1.1 Ce qu'on a appris

Un `float` en C++ n'est pas juste un nombre. C'est 32 bits organisés en trois champs selon la norme IEEE 754 :

$$\underbrace{s}_{1 \text{ bit}} \quad \underbrace{e_7 e_6 \dots e_0}_{8 \text{ bits}} \quad \underbrace{m_{22} m_{21} \dots m_0}_{23 \text{ bits}}$$
$$\text{valeur} = (-1)^s \times 1, m \times 2^{e-127}$$

4.1.2 L'implémentation

La clé est d'utiliser `memcpy` pour réinterpréter les octets d'un `float` en `uint32_t` sans conversion de valeur :

Listing 4.1 – `inspectFloat` : lire les bits d'un flottant

```
1 void NkMath::inspectFloat(float x) {
2     uint32_t bits;
3     // Copie les bits tels quels, sans conversion
4     std::memcpy(&bits, &x, sizeof(bits));
5
6     uint32_t sign      = (bits >> 31) & 0x1;    // bit 31
7     uint32_t exponent  = (bits >> 23) & 0xFF;    // bits 30-23
8     uint32_t mantissa  = bits & 0x7FFFFFFF;      // bits 22-0
9
10    logger.Info("Float_{0}: _signe={1}, _exposant={2}, _mantisse={3}",
11               x, sign, exponent, mantissa);
12 }
```

4.1.3 Ce que les tests ont révélé

Entrée	Signe	Exposant	Type
0.5f	0	01111110	Normal
1.0f/0.0f	0	11111111	$+\infty$
<code>sqrt(-1.0f)</code>	1	11111111	NaN
-0.0f	1	00000000	Zéro négatif

TABLE 4.1 – Cas remarquables de l’inspection IEEE 754

4.2 TP2 : Quand les additions deviennent mensongères

4.2.1 Le problème de la sommation naïve

Additionner un million de 0.1f ne donne pas exactement 100000.0f. Pourquoi ? Parce que 0.1 ne s’écrit pas exactement en binaire — exactement comme $1/3$ ne s’écrit pas en décimal.

Comparaison des sommes

Somme accumulee	: 100958	erreur de 958 !
Somme Kahan	: 100000	quasi parfait
Valeur réelle	: 100000.0	

4.2.2 L’algorithme de Kahan : compenser les erreurs

L’idée géniale de Kahan : garder une trace de l’erreur perdue à chaque addition et la rajouter au tour suivant.

Listing 4.2 – Sommation de Kahan

```

1 float NkMath::kahanSum(std::vector<float>& data) {
2     float sum = 0.0f;
3     float comp = 0.0f; // l'erreur accumulee
4     for (float val : data) {
5         float y = val - comp; // corriger avec l'erreur precedente
6         float t = sum + y;    // addition : perte de bits possible
7         comp = (t - sum) - y; // capturer les bits perdus
8         sum = t;

```

```
9     }  
10    return sum;  
11 }
```

4.3 TP3 : 33 tests pour valider tout ça

Ce TP consiste à écrire 33 assertions qui vérifient le comportement de `isFiniteValid`, `nearlyZero`, `approxEq` et `kahanSum`. L'enjeu : couvrir les cas limites (NaN, infini, zéro, grandes valeurs...).

33/33 assertions passent

Loic_TP3_ValidationFonctionsFloat [OK] 33/33 assertions

Chapitre 5

TPs 4 à 6 : Des vecteurs en 2D, 3D et 4D

5.1 TP4 : Vec2d — les fondations

Le vecteur 2D est le point de départ. Vingt tests valident ses opérations :

Opération	Formule	Tests
Dot(u,v)	$u_x v_x + u_y v_y$	6
Cross2D(u,v)	$u_x v_y - u_y v_x$	4
Normalized()	$v/\ v\ $	4
operator[]	accès par indice	5
static_assert	taille = 16 octets	1

TABLE 5.1 – Opérations de Vec2d

Une contrainte intéressante : la taille mémoire est vérifiée *à la compilation* :

```
1 static_assert(sizeof(Vec2d) == 16, "Vec2d doit faire 16 octets");  
2 // 2 doubles x 8 octets = 16 octets
```

5.2 TP5 : Vec3d et Gram-Schmidt

5.2.1 Le produit vectoriel

En 3D, le produit vectoriel $\mathbf{u} \times \mathbf{v}$ donne un vecteur perpendiculaire aux deux autres, suivant la règle de la main droite :

$$\mathbf{u} \times \mathbf{v} = \begin{pmatrix} u_y v_z - u_z v_y \\ u_z v_x - u_x v_z \\ u_x v_y - u_y v_x \end{pmatrix}$$

5.2.2 Gram-Schmidt : construire une base orthonormée

Étant donné 3 vecteurs quelconques, Gram-Schmidt produit 3 vecteurs unitaires et mutuellement perpendiculaires :

Listing 5.1 – Orthogonalisation de Gram-Schmidt

```

1 Vec3d ui = a.Normalized();
2 Vec3d vi = (b - Project(b, ui)).Normalized();
3 Vec3d wi = (c - Project(c, ui) - Project(c, vi)).Normalized();
4
5 // Verification : les 3 vecteurs sont orthogonaux deux a deux
6 assert(approxEq(Dot(ui, vi), 0.0)); // ui      vi
7 assert(approxEq(Dot(ui, wi), 0.0)); // ui      wi
8 assert(approxEq(Dot(vi, wi), 0.0)); // vi      wi

```

Ce test tourne sur 10 triplets aléatoires. Tous passent.

5.3 TP6 : Vec4d et la première image 3D

Le vecteur 4D permet la projection perspective : un point 3D (X, Y, Z) devient un point 2D par la formule :

$$x_{2D} = \frac{X}{Z}, \quad y_{2D} = \frac{Y}{Z}$$

Le résultat est une image `cube_TP6.ppm` — le premier rendu 3D du projet !

Chapitre 6

TPs 7 à 9 : Matrices, Rasteriseur et TRS

6.1 TP7 : La matrice 4×4 et son inverse

La matrice 4×4 est l'outil central du rendu 3D. Elle peut représenter n'importe quelle transformation géométrique (translation, rotation, mise à l'échelle).

Trois propriétés clés sont testées sur 10 matrices aléatoires :

1. $M \times I_4 = M$ (identité neutre)
2. $M \times M^{-1} = I_4$ (inverse)
3. Une matrice singulière n'est pas inversible (Inverse retourne false)

24/24 assertions

Loic_TP7_Mat4dInverseEtRotation [OK] 24/24 assertions

6.2 TP8 : Le cube tourne !

Le TP8 implémente un pipeline de rendu complet. Pour chaque frame :

$$\mathbf{p}_{\text{écran}} = \text{ProjectToScreen}(P \cdot V \cdot R \cdot \mathbf{v})$$

R Rotation du cube autour de l'axe Y

V Matrice de vue — position et orientation de la caméra

P Projection perspective — fov 45°, near 0.1, far 100

10 images sont générées : `frame_TP8_0.ppm` à `frame_TP8_9.ppm`.

6.3 TP9 : TRS et décomposition

La transformation TRS combine Translation, Rotation et Scale en une seule matrice :

$$M_{TRS} = T \cdot R_x \cdot R_y \cdot R_z \cdot S$$

La fonction `DecomposeTRS` fait l'opération inverse : à partir de la matrice, elle retrouve les composantes originales. 20 cas aléatoires sont testés.

Chapitre 7

TPs 10 et 11 : Quaternions et Animation

"Les matrices ça marche, mais les quaternions c'est élégant."

— Tout développeur 3D

7.1 TP10 : Les quaternions

7.1.1 Pourquoi les quaternions ?

Les matrices de rotation ont un problème connu : le *gimbal lock* (blocage de cardan), où deux axes de rotation peuvent s'aligner et faire perdre un degré de liberté. Les quaternions résolvent ce problème élégamment.

Un quaternion est de la forme :

$$q = w + xi + yj + zk, \quad \|q\| = 1$$

7.1.2 Ce qui est testé

Test 1 : Rotation de $\pi/2$ autour de Y transforme $(1, 0, 0)$ en $(0, 0, -1)$

```
1 Quat q1 = FromAxisAngle({0,1,0}, PI / 2.0f);
2 Vec3d j = Rotate(q1, {1,0,0});
3 // j doit valoir (0, 0, -1)
4 ASSERT_TRUE(std::fabs(j.z + 1.0) < kEps);
```

Test 2 : Aller-retour $q \rightarrow Mat3 \rightarrow q$ — 50 quaternions aléatoires

Test 3 : $q \times q^{-1} = \text{identité}$ — 50 quaternions aléatoires

103/103 assertions

Loïc_TP10_Quaternions [OK] 103/103 assertions

7.2 TP11 : SLERP vs LERP — animer proprement

7.2.1 Le problème de l'interpolation de rotations

Interpoler deux rotations ne se fait pas comme on interpole deux nombres. Si on utilise une interpolation linéaire (LERP) directement sur les composantes du quaternion, la vitesse de rotation n'est pas constante — le cube "accélère" et "ralentit" pendant l'animation.

7.2.2 SLERP : l'interpolation sphérique

SLERP (Spherical Linear intERPolation) interpole le long du grand arc sur la sphère unitaire des quaternions :

$$\text{Slerp}(q_1, q_2, t) = q_1 \left(q_1^{-1} q_2 \right)^t$$

Le résultat : une rotation à vitesse angulaire *constante*.

7.2.3 Ce que le code produit

Deux séquences de 60 frames sont générées :

- Slerp_frame_TP11_0.ppm à Slerp_frame_TP11_59.ppm — interpolation sphérique
- Lerp_frame_TP11_0.ppm à Lerp_frame_TP11_59.ppm — interpolation linéaire

La différence entre les deux méthodes devient visible vers le milieu de l'animation.

Chapitre 8

Bilan final : ce que j'en retiens

8.1 Résultats des tests

TP	Nom du test	Ce qui est validé	Assertions	Status
1	Loic_TP1	Décomposition IEEE 754	0 (affichage)	
2	Loic_TP2	Précision numérique	0 (logs)	
3	Loic_TP3	isFiniteValid, nearlyZero, approxEq, kahanSum	33	
4	Loic_TP4	Vec2d complet	19	
5	Loic_TP5	Vec3d + Gram-Schmidt	67	
6	Loic_TP6	Projection perspective cube	0 (image)	
7	Loic_TP7	Mat4d inverse + rotation	24	
8	Loic_TP8	Rasteriseur cube tournant	0 (frames)	
9	Loic_TP9	TRS + décomposition	60	
10	Loic_TP10	Quaternions	103	
11	Loic_TP11	Animation SLERP/LERP	0 (frames)	
Total			306	11/11

TABLE 8.1 – Résultats finaux — 11/11 tests, 306 assertions

Score final

```
Tests :      11 reussis, 0 echoues, 11 au total
Assertions : 306 reussies, 0 echouees, 306 au total
Taux succes : 100%
```

8.2 Les vraies leçons de ce projet

Au-delà des maths et du C++, ce projet m'a appris plusieurs choses concrètes :

1. **Les erreurs d'installation sont normales.** Chaque erreur avait une cause précise et une solution. La patience et la lecture attentive des messages d'erreur sont essentielles.
2. **La précision numérique en virgule flottante est un vrai sujet.** L'algorithme de Kahan n'est pas un détail académique — en VR/AR, des erreurs d'arrondi accumulées peuvent déformer les géométries de manière visible.
3. **Les quaternions ne sont pas si effrayants.** Une fois qu'on comprend qu'ils représentent des rotations sur une sphère à 4 dimensions, l'interpolation SLERP devient intuitive.
4. **Les tests unitaires sauvent des vies.** Sans les 306 assertions, beaucoup de bugs mathématiques subtils seraient passés inaperçus.
5. **Git est un ami précieux** — surtout quand la deadline approche et qu'on a besoin de pousser son code rapidement.

Dépôt *GitHub* :

<https://github.com/Tezz937/Nkentseu/tree/RihenAcademicSchoolBeginning>

Bibliographie

- [1] Teuguia Tadjuidje Rodolf Sédérís, *Jenga Build System v2.0.1*, Riheh, 2025. <https://github.com/RihenUniverse/Jenga>
- [2] Riheh Universe, *Nkentseu C++ Framework*, 2025. <https://github.com/RihenUniverse/nkentseu>
- [3] IEEE, *IEEE Standard for Floating-Point Arithmetic 754-2019*, 2019.
- [4] W. Kahan, *Further Remarks on Reducing Truncation Errors*, Comm. ACM, 8(1), 1965.
- [5] K. Shoemake, *Animating Rotation with Quaternion Curves*, SIGGRAPH, 1985.